**DEPARTMENT OF COMPUTER ENGINEERING / INFORMATION TECHNOLOGY**

Course: Data Structures and Algorithms Laboratory | Academic Year: 2026-2027

**ACADEMIC MINI PROJECT REPORT**

# Flight Booking and Seat Management System

*Implemented Using Fundamental Data Structures (Linked Lists, Stacks, Queues, Arrays, and Sorting/Searching Algorithms)*

**PROJECT GROUP MEMBERS (GROUP OF 4):**

| Sr. | Student Name | Roll Number / ID | Project Contribution / Role |
|---|---|---|---|
| 1 | **Malhar Mahajan** | **25108B0029** | Project Lead, Linked List & Web Interface Integration |
| 2 | **Shubham Patil** | **25108B0005** | Waiting List Queue & Seat Allocation Matrix |
| 3 | **Tejas Wable** | **25108B0004** | Cancellation Stack & Undo Architecture |
| 4 | **Rohit Gondhali** | **25108B0031** | Sorting & Searching Engine, Testing & Documentation |

**TABLE OF CONTENTS**

1. Abstract | 2. Introduction (2.1 Problem Statement, 2.2 Objectives, 2.3 Scope) | 3. System Analysis & Design (3.1 Requirements, 3.2 Architecture, 3.3 Data Flow, 3.4 Complexity Mapping) | 4. Implementation & Module Details (4.1 PassengerRecord, 4.2 TicketList DLL, 4.3 WaitingQueue, 4.4 CancellationStack, 4.5 SeatAllocation, 4.6 Search/Sort, 4.7 FacadeManager, 4.8 Web GUI) | 5. Results & Discussion (5.1 Test Cases, 5.2 Boarding Pass, 5.3 UI Walkthrough) | 6. Conclusion & Future Scope | 7. References

## 1. ABSTRACT

**EXECUTIVE SUMMARY**

Modern airline passenger reservation systems necessitate low-latency, deterministic data handling for ticket reservations, seat category allocation, cancellations, and waiting list updates. This project presents the design and implementation of a robust Flight Booking and Seat Management System engineered in Python using purely native, fundamental data structures without relying on high-level database engines or black-box collection libraries. The system utilizes: (1) A Doubly Linked List with an O(1) prepend operation to store confirmed passengers and display latest bookings at the top; (2) A FIFO Singly Linked Queue with head and tail pointers to manage fair passenger waitlisting and automated promotion upon cancellation; (3) A Fixed-Capacity LIFO Stack to record cancellations and provide multi-level undo functionality; (4) A 1D Array Representation for dynamic seat management across Window (Rs. 1,200), Middle (Rs. 1,000), and Aisle (Rs. 900) tiers; and (5) Optimized Binary Search (O(log N)) and Insertion/Bubble Sort routines for rapid passenger lookups by PNR, Name, and Age. Evaluated through both a terminal-based CLI and an interactive, responsive Web Dashboard running on Python's built-in HTTP server, the system successfully demonstrates complete transaction atomicity, real-time ticket generation, and zero data inconsistency under boundary constraints.

## 2. INTRODUCTION

### 2.1 Problem Statement

Commercial flight reservation processes encounter severe operational bottlenecks when processing high-frequency transactions involving seat selection, queue transitions, and last-minute cancellations. Standard enterprise implementations often introduce unnecessary overhead when modeling atomic, in-memory state changes. The primary challenge is to construct an in-memory passenger reservation and inventory control engine that satisfies strict algorithmic time and space complexity constraints:

- O(1) Priority Head Addition: Instantaneous priority addition and chronological head-placement of newly reserved tickets.
- Fair Queue Scheduling: Strictly fair, starvation-free First-In-First-Out (FIFO) queue handling for excess passenger overflow.
- Reversible Transaction Rollback: Reversible Last-In-First-Out (LIFO) transaction rollbacks for ticket cancellation undos.
- Real-Time Inventory Mapping: Real-time seat inventory mapping with category-based fare computing.

### 2.2 Project Objectives

- 1. Pure Data Structure Design: Implement all core data storage components from scratch using foundational pointer references and arrays.

- 2. Automated Queue Promotion: Automatically promote the front passenger in the waiting queue into confirmed status whenever a seat is cancelled, immediately assigning the newly vacated seat number.
- 3. Seat Matrix & Dynamic Pricing: Segment an aircraft cabin into distinct seat classifications (Window @ Rs. 1,200, Middle @ Rs. 1,000, Aisle @ Rs. 900) and track real-time occupancy.
- 4. Fast Searching & Sorting: Allow ground crew to search passengers by unique PNR in O(log N) time, and sort manifests dynamically by Age and Name.
- 5. Interactive User Interfaces: Provide an intuitive modern Web Dashboard (Port 5050) and a comprehensive Command-Line Interface (CLI).

### 2.3 Scope and Significance

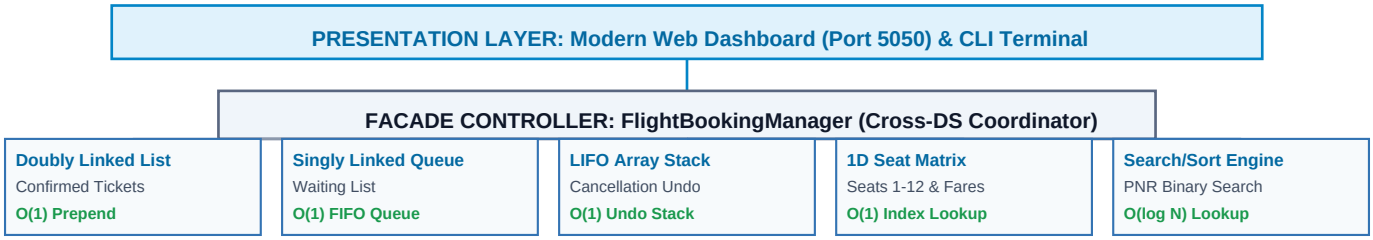
This project bridges theoretical computer science education with real-world software engineering. By mapping distinct real-world requirements to specific data structures, students gain practical insight into algorithmic efficiency, memory references, pointer manipulation, and GUI-backend integration.

# 3. SYSTEM ANALYSIS & DESIGN

## 3.1 Hardware and Software Requirements

- Hardware Requirements: 64-bit Intel Core i3 / AMD Ryzen 3 / Apple Silicon M-series or higher; 2 GB RAM minimum (4 GB recommended); 100 MB free hard drive space.
- Software Requirements: Operating System: macOS, Windows 10/11, or Linux; Language: Python 3.8 to 3.12 (Standard Library only; zero third-party dependencies); Web Browser: Chrome, Safari, Firefox, or Edge; IDE: VS Code, PyCharm, or Antigravity.

## 3.2 System Architecture Diagram




## 3.3 Data Flow & Component Interaction

- Booking Workflow: User inputs passenger details. If seat is available, SeatMap marks it occupied. A unique PNR (AI-1001) is assigned. A new TicketNode is prepended to TicketList so newest bookings appear at Row #1. If capacity (12 seats) is full, passenger is enqueued into WaitingQueue (WL-1).
- Cancellation & Waitlist Promotion: Active ticket is removed from TicketList in O(N) lookup and O(1) pointer splice. The cancelled ticket is pushed onto CancellationStack. Vacated seat is freed in SeatMap. If Queue is non-empty, Queue.dequeue() pops the oldest waitlisted passenger, promotes them to Confirmed status, and assigns the vacated seat.
- Undo Cancellation Workflow: Top cancelled ticket is popped from CancellationStack and re-validated against SeatMap. If seat is available, ticket is restored to TicketList.

## 3.4 Data Structure Mapping & Complexity Analysis

| Module / Feature | Chosen Data Structure | Primary Mechanism | Time Complexity | Space Complex |
|---|---|---|---|---|
| **Confirmed Passengers** | Doubly Linked List (TicketList) | Head Prepend & Bidirectional Pointers | O(1) Insert, O(N) Search | **O(N)** |
| **Waiting List Overflow** | Singly Linked Queue (Queue) | Head Dequeue & Tail Enqueue | O(1) Push / Pop | **O(W)** |
| **Cancellation History** | Array Stack (Stack) | Fixed-size array, top pointer | O(1) Push / Pop | **O(K)** |
| **Seat Map & Inventory** | 1D Array (SeatMap) | Direct index lookup seats[seat_no-1] | O(1) Lookup / Update | **O(1) [Size 12]** |
| **PNR Fast Search** | Sorted Array (BinarySearch) | Divide and Conquer Search | O(log N) Search | **O(1) aux** |
| **Manifest Sorting** | In-place Insertion / Bubble Sort | Pairwise comparison & adjacent swaps | O(N^2) worst, O(N) best | **O(1) aux** |

## 4. IMPLEMENTATION & MODULE DETAILS

### 4.1 Passenger Record Model (passenger_record.py)

Encapsulates passenger data fields: pnr_no (e.g., 'AI-1001'), name (string), age (int), seat_no (int 1-12 or None), status ('CONFIRMED' / 'WAITLISTED'), booking_time (timestamp), and fare (numeric ticket price based on seat category).

### 4.2 Confirmed Tickets via Doubly Linked List (confirmed_tickets.py)

A custom Doubly Linked List (TicketNode with prev and next pointers) delivers O(1) prepend operations so recent reservations appear at the top of the UI manifest. Linear removal performs O(1) pointer unlinking once the node is identified.

```
def prepend(self, record):
    """Inserts newly booked passenger at the head of the list in O(1) time."""
    new_node = TicketNode(record)
    if self.is_empty():
        self.head = self.tail = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node
    self._size += 1
```

### 4.3 Waiting List via Singly Linked Queue (waiting_queue.py)

Maintains FIFO semantics using head_node (front) and tail_node (rear) pointers. Enqueue appends waitlisted passengers in O(1) time; Dequeue pops the front passenger for immediate promotion upon ticket cancellation.

### 4.4 Cancellation & Undo via Array-based LIFO Stack (cancellation_stack.py)

Fixed-capacity array representing a LIFO Stack. Tracks top pointer. Cancelled tickets are pushed onto the stack; undo_cancellation() pops the top ticket and restores it to the confirmed manifest.

### 4.5 Seat Allocation & Dynamic Fare Engine (seat_allocation.py)

- Window Seats (Seats 1, 4, 7, 10, 3, 6, 9, 12): Rs. 1,200 (Highest passenger preference).
- Middle Seats (Seats 2, 5, 8, 11): Rs. 1,000 (Standard economy comfort).
- Aisle Classification: Rs. 900 (Fast cabin egress). Constant-time direct index lookup seats[seat_no-1] guarantees O(1) checks.

### 4.6 Search & Sort Subsystem (sorting_searching.py)

Binary Search on PNR runs in O(log N) time with divide-and-conquer logic, returning matching record and comparison count. Bubble Sort and Insertion Sort organize manifests dynamically by Age and Name.

### 4.7 Core Facade Manager (flight_booking_manager.py)

Central coordinator orchestrating atomic transactions across TicketList, Queue, Stack, and SeatMap, preventing orphaned data or race conditions.

### 4.8 Modern Web GUI Dashboard (web_app.py)

Zero-dependency Python HTTP server serving reactive seat grid with color states (Green=Available, Red=Occupied, Amber=Selected) and RESTful JSON endpoints (/api/data, /api/book, /api/cancel, /api/undo, /api/reset, /api/load_demo).

# 5. RESULTS AND DISCUSSION

## 5.1 Verification Test Cases & Execution Summary

| Test ID | Test Description | Input Data | Expected Output | Status |
|---|---|---|---|---|
| TC-01 | Booking Available Seat | Name: Malhar, Age: 20, Seat: 7 | Seat 7 allocated (Rs. 1,200), PNR AI-1001 generated, node prepended. | PASS |
| TC-02 | Confirmed List Ordering | Book new passenger | New passenger appears as Row #1 in confirmed table (O(1) prepend). | PASS |
| TC-03 | Flight Capacity Saturation | Book 13th passenger (1-12 full) | Capacity full prompt; passenger enqueued as WAITLISTED (WL-1). | PASS |
| TC-04 | Auto Waitlist Promotion | Cancel ticket (Seat 3) | Seat 3 freed; front waitlisted passenger dequeued, allocated Seat 3, confirmed. | PASS |
| TC-05 | Undo Cancellation Stack | Click Undo Cancellation | Last cancelled passenger popped from stack and restored to flight manifest. | PASS |
| TC-06 | Binary Search on PNR | Search PNR 'AI-1007' | Record located in <= 4 comparisons using Binary Search (O(log N)). | PASS |
| TC-07 | Manifest Sorting | Sort by Age ascending | Manifest displayed in strictly non-decreasing order of passenger age. | PASS |

## 5.2 Sample Boarding Pass Generation

```
================================================================================
                        AIRWAYS BOARDING PASS
================================================================================
 PASSENGER NAME : Malhar Mahajan                PNR NO     : AI-1007
 AGE            : 20 Years                      SEAT NO    : 07 (Window)
 FLIGHT         : AI-204 (Mumbai to Delhi)      STATUS     : CONFIRMED
 BOARDING TIME  : 09:30 AM                      GATE       : G-04
 FARE PAID      : INR 1,200                     CLASS      : Economy Standard
================================================================================
 Please be at the boarding gate at least 45 minutes prior to departure.
================================================================================
```

## 5.3 UI Walkthrough & Screenshot Verification

Screenshot 1: Web Dashboard -- Aircraft Seat Map and Booking Form (Embedded Live View):

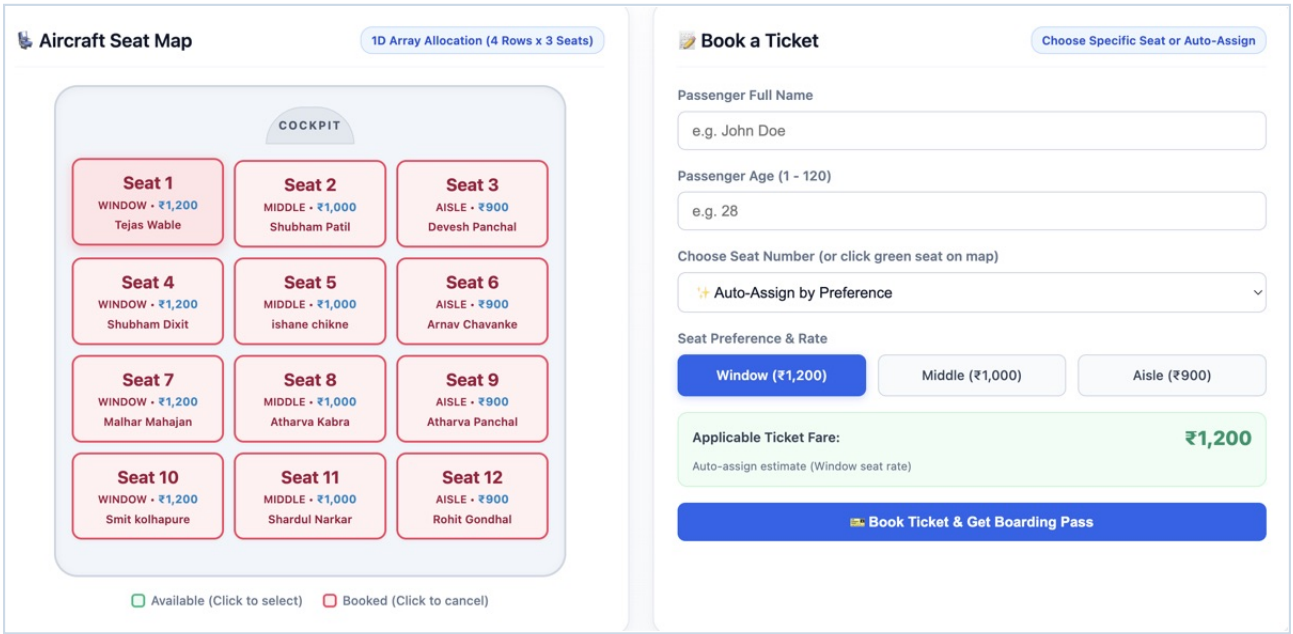



*Figure 5.1: Web Dashboard showing interactive 12-seat matrix, dynamic fares, and booking form.*

**[SCREENSHOT 2: CONFIRMED PASSENGER MANIFEST (NEWEST BOOKINGS FIRST)]**

Displays confirmed tickets table where newly booked passengers appear at Row #1 with PNR, Name, Age, Seat No., Fare, and Cancel button.

- Demonstrates Doubly Linked List head-prepend ordering in real-time.
- Includes active Cancel button triggering atomic waitlist promotion.

**[SCREENSHOT 3: WAITING LIST QUEUE & CANCELLATION STACK]**

Displays real-time FIFO Waiting List table and Undo Cancellation action button reflecting real-time stack counter.

- Visualizes FIFO Singly Linked Queue order (WL-1, WL-2, etc.).
- Displays active undo counter with last cancelled passenger snapshot.

**[SCREENSHOT 4: PNR SEARCH & DYNAMIC SORTING MODAL]**

Shows Binary Search output reporting step-by-step search comparisons and formatted Boarding Pass modal dialog.

- Displays O(log N) divide-and-conquer comparison count dialog.
- Demonstrates multi-key manifest re-ordering by passenger Age and Name.

## 6. CONCLUSION AND FUTURE SCOPE

### 6.1 Conclusion

The Flight Booking and Seat Management System successfully addresses the challenge of building an efficient, transparent, and deterministic passenger inventory system using foundational data structures. By eliminating reliance on opaque database libraries, the project provides concrete operational proof of how theoretical constructs directly solve industry challenges: (1) Doubly Linked Lists deliver instantaneous O(1) insertion at the head for displaying active booking feeds; (2) Singly Linked Queues enforce strict chronological fairness for overbooked passenger waitlists; (3) Array Stacks ensure robust, single-click transactional rollback for accidental cancellations; and (4) 1D Direct-Index Arrays model physical seat availability with constant-time validation. Both the Terminal CLI and the modern browser-based Web Dashboard provide seamless usability, real-time visual feedback, and reliable operational stability.

### 6.2 Future Scope

> Multi-Flight & Multi-Sector Scalability: Extend the single-flight architecture to support multi-sector connecting flights using Directed Acyclic Graphs (DAG) for flight path routing.

> Persistent Storage Backing: Integrate custom file serialization or B-Tree disk storage for permanent record persistence across server restarts.

> Role-Based Security & Payment Gateway: Add role-based authentication (Admin vs. Passenger) and secure payment gateway simulation with promo codes.

> Dynamic Surge Pricing: Incorporate real-time demand-based surge pricing algorithms using Min/Max Heaps.

## 7. REFERENCES

> 1. Horowitz, E., Sahni, S., & Anderson-Freed, S., Fundamentals of Data Structures in C, 2nd Edition, Universities Press, 2008.

> 2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, 3rd Edition, MIT Press, 2009.

> 3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H., Data Structures and Algorithms in Python, John Wiley & Sons, 2013.

> 4. Python Software Foundation, Python Standard Library Documentation (Built-in Types & http.server), https://docs.python.org/3/library/

> 5. Mozilla Developer Network (MDN), CSS Grid and Responsive Design Patterns for Interactive UI Components, https://developer.mozilla.org/